LINEAR REGRESSION

with One Variable

AI and ML Course — Lecture 2

- 📐 Regression Equation
- 📉 Residuals
- 📊 Fitted Values
- 🔢 Least Squares

Prerequisites: ML Overview · Data Types (Categorical, Numerical, Ordinal)

What IS Regression, Really?

> *"Given what I know about X, what can I PREDICT about Y?"*



House Prices

X (Input): Size (sq ft)
Y (Output): Price ($)



Running Speed

X (Input): Training hours/week
Y (Output): Race time (min)



Ice Cream Sales

X (Input): Temperature (°C)
Y (Output): Units sold

🔑 **Regression finds the BEST LINE through the data to make predictions.**

The Regression Equation

$$\hat{Y} = b_0 + b_1 \cdot X$$

$\hat{Y}$

Predicted Output

The value the model predicts (Y-hat). E.g. predicted house price.

b_0

Intercept

Where the line crosses the Y-axis. The prediction when X = 0.

b_1

Slope

How much Y changes for every 1-unit increase in X.

💡 **Real Example: PEFR = 424.58 – 4.185 × (Years of Dust Exposure) → $b_0 = 424.58$, $b_1 = -4.185$**

Fitted Values — "The Predictions"

What are Fitted Values?

When we plug each real X value into our equation, we get the model's guess for Y.

These "guesses" are called FITTED VALUES, written as $\hat{Y}_i$.

Formula:

$$\hat{Y}_i = \hat{b}_0 + \hat{b}_1 \cdot X_i$$

Note: $\hat{b}_0$ and $\hat{b}_1$ use a hat (^) to show they are ESTIMATED from data, not exact.

House Price Example

Model: Price = 50,000 + 200 × Size

Size (sq ft)	Actual Price (Y)	Fitted ($\hat{Y}$)
800	$210,000	$210,000
1000	$252,000	$250,000
1200	$298,000	$290,000
1500	$355,000	$350,000

Fitted values are on the regression line. Real values are scattered around it.

Fitted values = the model's best guess using the learned equation.

Residuals — "The Errors"

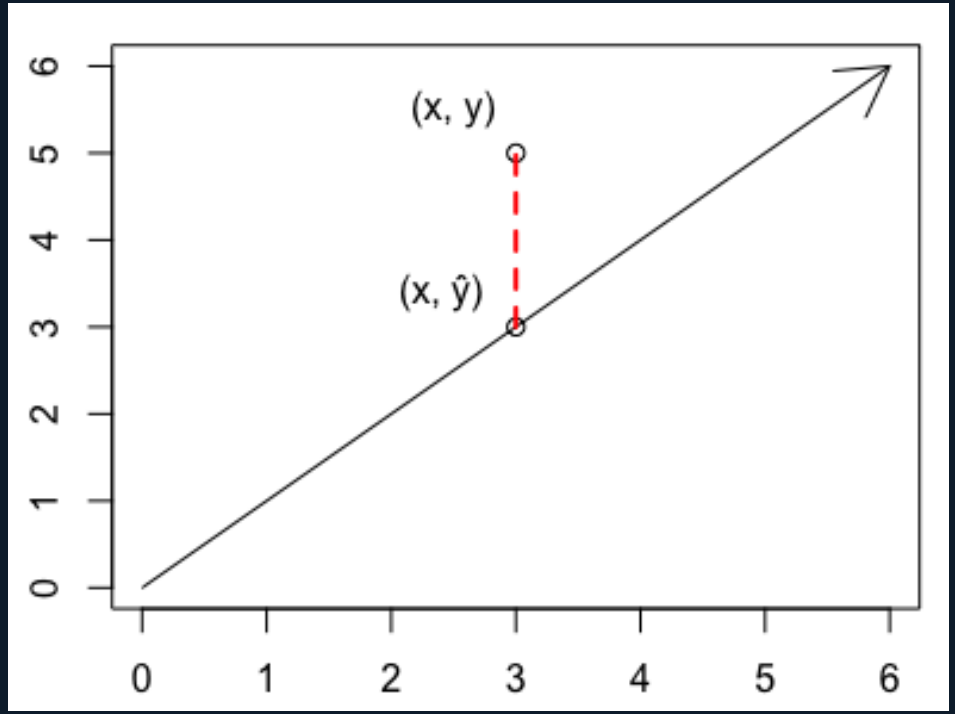
Definition

A residual is the vertical distance between the ACTUAL data point and the PREDICTED point on the line.

$$e_i = Y_i - \hat{Y}_i$$

Positive residual → Model UNDERESTIMATED (real > predicted)
Negative residual → Model OVERESTIMATED (real < predicted)
Zero residual → Perfect prediction! (rare)

Visual Intuition



🔑 **Small residuals = good model. Large residuals = poor fit.**

Residuals — "The Errors"

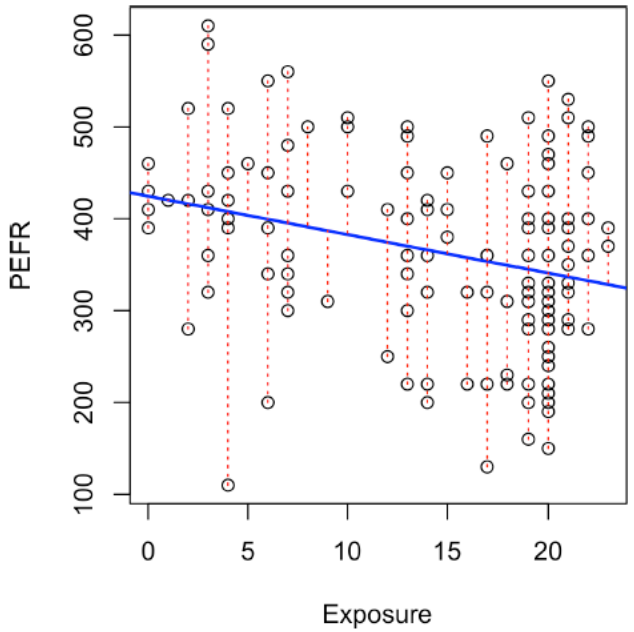
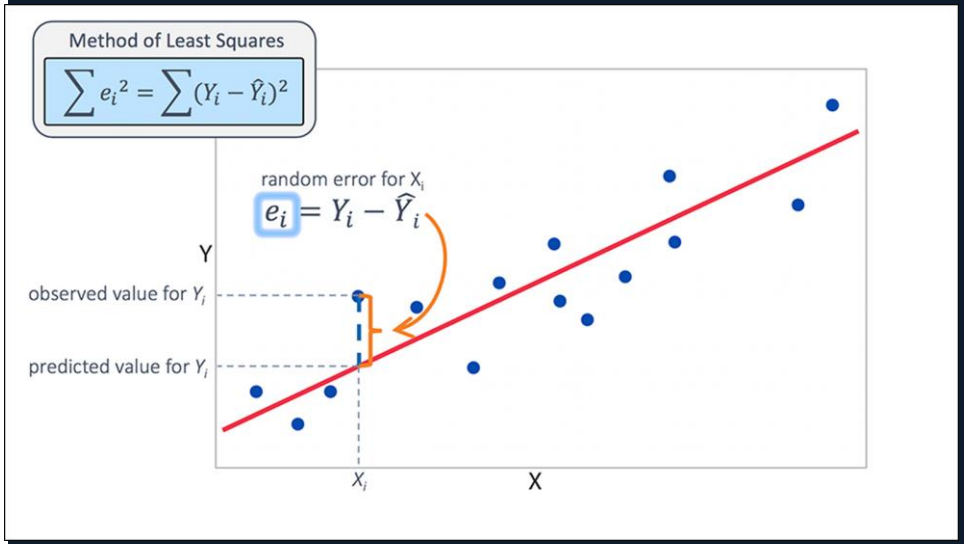


Figure 4-3. Residuals from a regression line (to accommodate all the data, the y-axis scale differs from Figure 4-2, hence the apparently different slope)



Least Squares — How the Line is Fit

1

Compute Residuals

For each point:
$e_i = Y_i - \hat{Y}_i$
Some are positive, some negative.

2

Square Them

Squaring removes negatives and penalises large errors more.
$(e_i)^2 = (Y_i - \hat{Y}_i)^2$

3

Sum

$$RSS = \sum_{i=1}^{n} \left(Y_i - \hat{Y}_i\right)^2$$

$$= \sum_{i=1}^{n} \left(Y_i - \hat{b}_0 - \hat{b}_1 X_i\right)^2$$

Find $\hat{b}_0$ & $\hat{b}_1$ that make RSS as small as possible.

$$b_1 = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

$$b_0 = \frac{\sum y - b_1 \sum x}{n}$$

Example

🏠 **Example: Predict house price from size. We have 4 data points:**

X (Size sqft)	Y (price $k)
800	210
1000	250
1200	310
1500	370

Home Work

b_1 = ?
b_o = ?

Final Model:

Price = bo + b1 × Size

For Size = 1100 sqft:
Price = b0 + ? × 1100 = ?

Implementing Regression in Python

regression.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Data: Example of a simple dataset for linear regression
X = np.array([[800],[1000],[1200],[1500]]) # Independent variable (e.g., area in sq ft)
y = np.array([210,250,310,370])          # Dependent variable (e.g., price in thousands)

# Train the model
model = LinearRegression() # Initialize the Linear Regression model
model.fit(X, y)            # Fit the model to the data

# Coefficients
print('b0:', model.intercept_) # Print the intercept (b0)
print('b1:', model.coef_)      # Print the coefficient (b1)

# Predict
y_hat = model.predict(X)       # Predict y values using the trained model
residuals = y - y_hat          # Calculate the residuals (actual y - predicted y)
residuals
```

Output

```
b0: 22.15
b1: [0.234]
Residuals: [0.93, -5.80, 7.48,2.62]
```

Key Methods

.fit(X, y) — trains the model
.predict(X) — get fitted values
.intercept_ — b_0
.coef_ — b_1

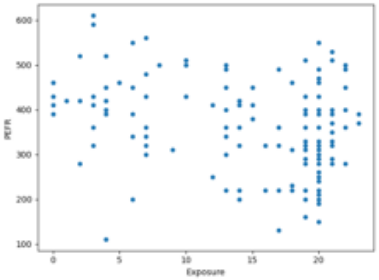
Run this in Google Colab — free GPU, no installation needed!

Colab code for regression

```
lung = pd.read_csv(LUNG_CSV)

lung.plot.scatter(x='Exposure', y='PEFR')

plt.tight_layout()
plt.show()
```



The intercept, or b0, is **424.583** and can be interpreted as the predicted **PEFR for a worker with zero years exposure.** The regression coefficient, or **b1**, can be interpreted as follows: for each additional year that a worker is exposed to cotton dust, the worker's PEFR measurement is reduced by **-4.185.**

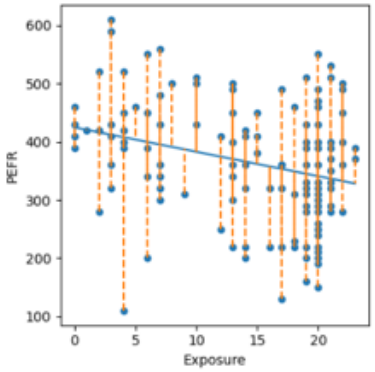
Fitted Values and Residuals

The method `predict` of a fitted *scikit-learn* model can be used to predict new data points.

```
fitted = model.predict(lung[predictors])
residuals = lung[outcome] - fitted
```

```
ax = lung.plot.scatter(x='Exposure', y='PEFR', figsize=(4, 4))
ax.plot(lung.Exposure, fitted)
for x, yactual, yfitted in zip(lung.Exposure, lung.PEFR, fitted):
    ax.plot((x, x), (yactual, yfitted), '--', color='C1')

plt.tight_layout()
plt.show()
```




```
predictors = ['Exposure']
outcome = 'PEFR'

model = LinearRegression()
model.fit(lung[predictors], lung[outcome])

print(f'Intercept: {model.intercept_:.3f}')
print(f'Coefficient Exposure: {model.coef_[0]:.3f}')
```

```
Intercept: 424.583
Coefficient Exposure: -4.185
```

```
fig, ax = plt.subplots(figsize=(4, 4))
ax.set_xlim(0, 23)
ax.set_ylim(295, 450)
ax.set_xlabel('Exposure')
ax.set_ylabel('PEFR')
ax.plot((0, 23), model.predict(pd.DataFrame({'Exposure': [0, 23]})))
ax.text(0.4, model.intercept_, r'$b_0$', size='larger')

x = pd.DataFrame({'Exposure': [7.5,17.5]})
y = model.predict(x)
ax.plot((7.5, 7.5, 17.5), (y[0], y[1], y[1]), '--')
ax.text(5, np.mean(y), r'$\Delta Y$', size='larger')
ax.text(12, y[1] - 10, r'$\Delta X$', size='larger')
ax.text(12, 390, r'$b_1 = \frac{\Delta Y}{\Delta X}$', size='larger')

plt.tight_layout()
plt.show()
```

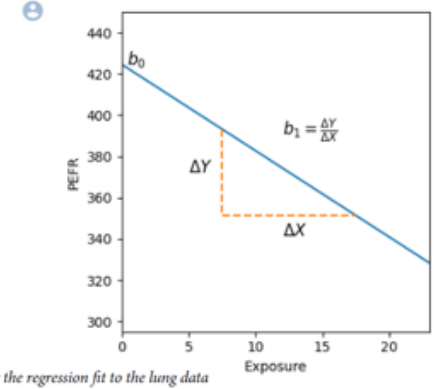


Figure 4-2. Slope and intercept for the regression fit to the lung data

How to Interpret a Regression Model

Real Study: Cotton Dust Exposure & Lung Function (PEFR)

$$PEFR = 424.58 - 4.185 \times (\text{Years of Exposure})$$

Interpreting b_0 (Intercept = 424.58)

If a worker has ZERO years of exposure to cotton dust, their predicted PEFR is 424.58 L/min.

This is the baseline lung function.

⚠️ *X=0 may not always be realistic — it's a mathematical baseline.*

Interpreting b_1 (Slope = −4.185)

For every 1 additional YEAR of cotton dust exposure, a worker's PEFR decreases by 4.185 L/min.

Negative slope = harmful relationship!

📊 *This is the most important number in a regression model.*

🔑 **Always interpret BOTH b_0 and b_1 in context of the real-world problem.**

SUMMARY

What We Covered Today in Lecture 2

1	**Regression Equation**	$\hat{Y} = b_0 + b_1X \rightarrow$ line that predicts Y from X
2	**Fitted Values**	Model's predictions ($\hat{Y}_i$) using learned b_0 & b_1
3	**Residuals**	$e_i = Y_i - \hat{Y}_i \rightarrow$ vertical gap between real & predicted
4	**Least Squares**	Minimise RSS = $\Sigma(e_i)^2$ to find the BEST line
5	**Python Implementation**	sklearn LinearRegression: .fit(), .predict(), .coef_

🚀 **Next Session: Multiple Linear Regression — predicting with MORE than one variable!**